Mining Software Repositories for the Characterization of Continuous Integration and Delivery

Gabriel Augusto Destro, Breno Bernard Nicolau de França
State University of Campinas (UNICAMP)
gabrielaugustodestro@gmail.com,breno@ic.unicamp.br

ABSTRACT

Continuous Integration (CI) and Delivery (CD) are software development practices increasingly adopted in industry, which aiming at ensuring quality and stability, and making the whole process more efficient and less error-prone. Software engineers that incorporate these practices may have difficulties trying to get an overview of the impact of adopting them and tracking progress. Therefore, performing analysis regarding the level of the application of CI/CD practices in a software repository can help to understand how such practices are adopted, which can result in a better characterization of their benefits and limitations. To support this analysis, we developed Garimpeiro, a web application that supports the characterization of open source repositories hosted on GitHub regarding the level of adoption of CI/CD practices. This paper describes how the tool extracts data from GitHub repositories by showing the execution workflow, high-level architecture, and involved assumptions to define metrics. Finally, this paper also includes running examples of the tool, which demonstrates its capabilities and feasibility.

CCS CONCEPTS

• **Software and its engineering → Software libraries and repositories**; **Agile software development**.

KEYWORDS

Mining Software Repositories, Continuous Integration, Continuous Delivery

ACM Reference Format:
Gabriel Augusto Destro, Breno Bernard Nicolau de França. 2020. Mining Software Repositories for the Characterization of Continuous Integration and Delivery. In *34th Brazilian Symposium on Software Engineering (SBES '20), October 21–23, 2020, Natal, Brazil.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3422392.3422503

1 INTRODUCTION

Continuous Integration (CI) and Delivery (CD) are software development practices that involve a cycle of developing, integrating, testing and delivering software in a consistent, regular and automated manner [2]. These practices have been increasingly adopted in industry, aiming at ensuring quality and stability, and making the whole process more efficient and less error-prone.

From the technical perspective, these practices are applied using a deployment pipeline, which is an automated tool chain for the steps involved in the development, building, tests and delivery of an application. However, this pipeline varies from project to project, since different procedures and tools are used according to the adopted technologies, platforms and application domain, and the specific needs of each organization [3].

The first step in a deployment pipeline involves detecting a repository change or an explicit request. It usually starts by listening to the source control repositories. GitHub is the most popular platform that makes use of the git source control system to host the source code of public and private projects. It also provides a REST API to retrieve project-related data that could not just be obtained through the project source code. In addition, the project repository contains several pieces of evidence on which are the adopted technologies for automated building, testing, and other tasks related to the deployment pipeline. These indicatives could be mined for analysing what are the technologies and tasks executed by the project team in terms of CI/CD along with project progress information.

Therefore, performing an analysis regarding the level of the application of CI/CD practices in a repository can help to understand how such practices are adopted, which can result in a better characterization of their benefits and limitations. Besides, project leaders can use this information for planning and improving their CI/CD procedures and coordination. Furthermore, such a tool can then support future studies targeting CI/CD by automating data collection through the mining of open source repositories.

In this paper, we present the Garimpeiro tool. It supports the characterization of projects hosted in GitHub with respect to their adoption of CI/CD practices through gathering evidence on CI/CD relevant information, from project metadata and repository, regarding tooling and procedures, i.e., how continuous the integration (and ultimately, delivery) practices can be, based on this data. Currently, Garimpeiro works better for the Java ecosystem. However, it can be extended for other platforms. Information regarding procedures and processes running outside the repository is out of scope as it is unfeasible to extract.

This characterization can help software engineers and other kind of users to analyse and have a clearer understanding of the current level of adoption of CI/CD practices in a project, which can lead to the improvement of the process by the enhancement of procedures and the addition of new tools. The data collection provided can also support studies that rely on extracting data from open software repositories regarding these practices.

The remaining sections are organized as follows. Section 2 presents the theoretical foundation. Section 3 presents the tool main features,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBES '20, October 21–23, 2020, Natal, Brazil
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8753-8/20/09…$15.00
https://doi.org/10.1145/3422392.3422503

target users, architecture and an execution example using a real open source repository. Section 4 shows how Garimpeiro can be used in a comparison scenario for two repositories. Finally, Section 5 presents the final remarks.

2 BACKGROUND

Humble and Farley [2] not only define the CI/CD practices, but also explain how they can be used in an effective manner through the application of patterns and good practices. The authors define these as software engineering practices, in which the developers build, execute, test and deliver software in a regular, consistent and automated manner. Therefore, these practices encompass a meaningful part of the software development cycle.

So, the authors describe the deployment pipeline as an automated implementation of the procedures involved in the development, build, test and delivery of an application. However, the deployment pipeline varies from company to company [4]. Thus, although each team has the same goals and guidelines, each one should adapt the pipeline in a way to fulfill their necessities. The pipeline works in such a way that each change made in the source code, environment, or data, triggers the creation of a new pipeline instance.

Humble and Farley [2] describe the "anatomy" of the deployment pipeline in its canonical form (Figure 1). Therefore, the characterization is important to understand the operation of these practices and then, not only determine which information should be obtained from repositories, but also understand the involved metrics that can characterize the level of application of CI/CD practices.

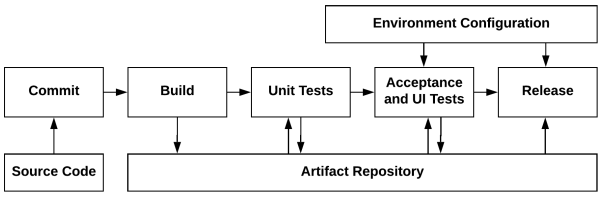

Figure 1: Deployment pipeline anatomy (adapted from [2]).

Munaiah and colleagues [5] present Reaper, a tool created for evaluating open source software hosted on GitHub in relation to the adoption of good software engineering practices. The Reaper tool was used as reference to comprehend the way used to retrieve data from these repositories, as the use of the GitHub REST API and the repository cloning. Some of the data gathered were the same in both works, for instance, the identification of the usage of tools for the management of continuous integration e test coverage.

Although the Reaper was used as reference during the development of this tool, it's important to take into account that both tools have different proposals. While Reaper proposes to gather general data related to the adoption of software engineering practices, to then rank each repository into planned or unplanned. The Garimpeiro tool searches for evidence of continuous delivery and integration practices in these repositories, not making any type of classification. Therefore, although both tools have similar ways of retrieving data, each one uses them in different ways, being that the Reaper has a more general proposal.

Finally, Mäkinen and colleagues [3] conducted a research with several software development organizations from different fields. As result, the authors characterized how these organizations customize their toolchains, considering the different stages of the software development process, in order to achieve CD benefits. Although it is important to notice that not all stages described in the toolchain are included in the deployment pipeline, and some tools can be related to more than one stage. Also, not all adopted tools can be detected through the analysis of the GitHub repository, since some of them leave no traces of usage in the repository.

We used this information to propose a reference deployment pipeline for the Garimpeiro tool (Figure 2), organized into stages and the related tools.

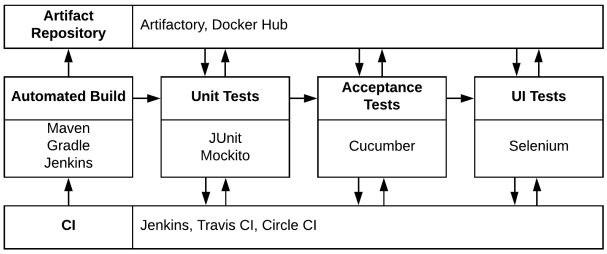

Figure 2: Pipeline with supported tools.

3 GARIMPEIRO TOOL

This section describes the Garimpeiro tool in terms of its main features, target users, and architecture. The tools is publicly available in the Gitlab repository [1]. Also, a running example illustrates how the tools work to accomplish its goals.

3.1 Potential Users and Features

As the Garimpeiro tool aims at characterizing software projects by looking for evidence on the adoption of CI/CD practices through mining software repositories, we understand its potential users could be both software engineering researchers and practitioners. These users may share common interest in terms of relevant information for CI/CD in the software repositories hosted in GitHub.

Software engineers can use the tool to make different types of analysis, which can help determine what kind of support they currently have for CI/CD practices, as well as using collected information to have insights on how they could achieve higher maturity on those practices by enhancing their procedures, adopting additional tools, and shortening delivery frequencies. On the other side, researchers may use this tool to support data collection on different kinds of studies relying on open data information.

Garimpeiro's main feature is the information retrieval from a GitHub repository to provide indications of use of CI/CD practices. For this, the tool gets data related to the frequency in which the repository is updated as commits and releases are performed. It also obtains information about the usage of tools that can be related to a certain deployment pipeline stage, along with information that

[1] Gitlab repository: https://gitlab.ic.unicamp.br/laser/garimpeiro

can give evidence about the project test coverage. Given an URL and a GitHub authentication token, the tool extracts the data from the repository, that then is compiled and presented to the user as a report. The extracted information can be visualized as a Web report or downloaded as a file (JSON format) so that one can use their own scripts and tools to process several files to perform custom data analysis, as there is no built-in comparison feature yet.

From the git source control system, the tool retrieves information related to commits, tags and releases. Total values are extracted, along with its frequency, represented by the mean number of contributions each day and month, the mean time between them and respective standard deviations.

For commits, the tool accounts for all the commits in all branches. With this information is possible to have an idea about the frequency in which the project codebase is incremented. Although it is not possible the say exactly when the pipeline execution started, as this information is not available in GitHub, it is a common practice to start a new instance of the pipeline each commit in specific branches, like the master branch, as feature branches generally don't fire new pipeline instances, when teams follow a branch per feature approach [1]. Therefore, a higher commit frequency may indicate that the application is tested more frequently in the case which unit tests are executed. A side effect of this way of obtaining commits is that merge and release commits are also counted. However, we understand as a reasonable assumption that, for most cases, these types of commit are a smaller percentage of the total number of commits, since our intention is to provide an estimation instead of an absolute value.

Garimpeiro obtains release data both from the releases section in GitHub and from repository tags. We cannot ensure a tag is related to a release, but it is also a good practice. This decision was made because there are cases that releases are not registered in the GitHub releases section, or cannot be obtained through the GitHub API. The tags are also retrieved from all the repository branches. Distinguishing tags that are and are not releases is a complex task, since it depends on the repository branching model, for instance, releases can be made on the master branch or in another release branch. It is still possible to use a new branch for each release, allied to that, there is no single convention for naming release tags. This set of factors makes it difficult to identify precisely if a tag is related to a release or not, but can be considered as an approximation.

Another consideration is that not all releases are associated with a software delivery. Some of them may be a documentation release associated with a software release, or even a release candidate. Therefore, a release in Garimpeiro is considered to be any kind of delivery that may or may not be revoked.

The tool can also detect, through the analyzes of the source and configuration files, the adoption of tools and frameworks that may be related to some stage of a deployment pipeline, such as automated build tools and unit tests. The tools are categorized according to the canonical pipeline (Figure 2).

Through the identified testing tools, it is also possible to estimate the level of test automation in the repository. For this, we calculate the proportion of the amount of test code files detected that makes use of a supported testing framework and the total amount of source files in the project repository. This information is then categorized for each test level and framework.

Garimpeiro also extracts data from the GitHub issue tracker. Issues are separated according to status (open or closed), so that a percentage of completion can be determined, along with the mean time between its creation and completion. We assume it as an estimate for the Lead Time. Garimpeiro also categorizes issues between features and bugs according to the issue tags or labels.

It is important to mention that, as the tool uses GitHub to extract the data, information regarding procedures and processes running outside the repository is unfeasible to extract. Despite that these procedures and processes are often supported by tools, these are usually external with no or one-way integration.

3.2 Architecture and Technologies

The tool executes the data collection and analysis in a sequential way (Figure 3). First, the repository is cloned locally. Second, Garimpeiro gathers git repository data on commits, tags, and badges in the "Readme" file. Third, through the GitHub REST API, the tool retrieves information usually obtained through GitHub repository web pages, such as topics, issues, releases, and contributors. Next, Garimpeiro searches the source code to detect tools and frameworks used to automate pipeline tasks, as build/dependency managers, testing frameworks, and CI servers. Finally, the data is assembled in a web report and also converted to JSON for download. All the extracted information is available in GitHub via the repository and the REST API combined.

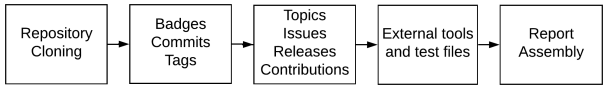

Figure 3: Mining process for Garimpeiro.

The high-level architecture of the Garimpeiro tool is present in Figure 4, including the main components and their dependencies. The WebClient, which is the component the user interacts with, performs requests to be processed by a central Controller component. This, in turn, connects to the repository using the GitRepository component and delegates the responsibility of building the CI/CD map for the MapModel component.

The latter uses the GitRepository and GitHubAPIClient components for raw data collection. The collected information is organized into Entities, representing issues, commits, tags, releases, contributors. The Metrics component is responsible for categorizing information accordingly and calculating relevant metrics like percentages, means and standard deviations based on Entities information. The PipelineTools component scans the cloned repository searching for files that provides indications on the usage of certain tools supported that can be related to the use of CI/CD practices.

The data gathering is performed in two steps. First, the GitRepository component clones the repository so that some data can be extracted locally without multiple calls to the GitHub REST API, saving requests and improving performance. In this step, the tool collects information related to commits, releases, tags and badges that remain stored in memory until the end of the process. The cloned repository files are later used by another module to retrieve

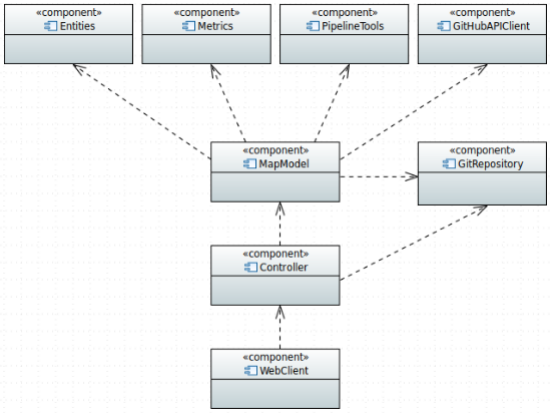

Figure 4: Garimpeiro's Architecture

data about test files and external tools that are related to some stage of the deployment pipeline. Regarding the implementation of this step, we use the Jgit library to clone the repository, since it already maps and handles the internal structure of git repositories.

In the second step, through the GitHubAPIClient module, we use the GitHub REST API, which is an API that returns project metadata available only in the GitHub repository web services. Therefore, this step retrieves information from topics and issues. As the GitHub API has a limitation on the number of requests that can be made per hour (60 requests for unauthenticated users and 5000 for authenticated users), it was used only to retrieve data that could not be obtained through the repository cloned. Although the number of requests available can grow according to the number of contributors depending on the organization, the tool was designed to minimize the number of requests, since the requests are limited and fetching data in the machine is faster than through the web.

Information on the usage of tools and frameworks are obtained by scanning files in the repository. This process is made by the PipelineTools component, after all the data from the other two modules have been retrieved. The usage of CI tools like Jenkins and Travis CI are detected by searching for specific files in the repository, for instance, having a file named "Jenkinsfile" indicates that the tool might be used. This indicates that a tool related to the CI pipeline stage was detected. Similarly, the same is done for the build automation and artifact integration tools, being then categorized in the correct pipeline stage.

For other types of frameworks, like Cucumber, it is searched for files with specific extensions. But for most of the supported frameworks, like JUnit and Mockito, the way used to detect is verifying each one of the source files and scanning for an import declaration of each of these frameworks. This is done searching for all the files with the programming language extension, like ".java", for instance. Then, each import declaration is analysed and, once the application detects an import of a particular framework, it considers that this framework is being used. Tools supported are selected from [3], in which the authors characterize commonly used tools for different pipeline stages. For the moment, only the Java programming language is supported.

Finally, the data is then categorized by the Metrics component. Relevant percentages, means and standard deviations are calculated for the commits, releases, tags, issues and test files. The data is also divided into days and months according to its relevance and separated according to labels ("bug" or "feature") in the case of issues, if a label that indicates this classification is found.

The Garimpeiro tool uses a Spring Boot back-end that processes the WebClient requests. After processing the repository mining, the results are exhibited in JSP pages (Java Server Pages).

3.3 Running Example

Figure 5 shows an execution example of the tool, the used repository was the Google's ExoPlayer. After inputting the repository URL and the GitHub authentication token, the tool starts the extraction of the data, which can take a few minutes according to the user's internet connection and the size of the repository.

After concluding the mining process, the report is exhibited. Clicking the "Download JSON Report" link downloads the same report parsed as a JSON file. Users can process this file using third party software, developed by them, in a way to compare data to several repositories simultaneously.

From the generated report, the user can identify the number of contributions (6670) and contributors (144), which gives an idea about the community involvement and size. Although no badges are found in its "Readme" file, some GitHub topics are. These indicate this is a media player Android application written in Java.

From the repository files, the Gradle tool can be identified, which is a famous build and dependency management tool in the Android apps development ecosystem and offered in the AndroidStudio IDE by default. Also, two frameworks classified as unit testing frameworks were identified: JUnit and Mockito. Garimpeiro also obtained the total number of test files, this data is used along the total number of source code files to calculate the percentage of test files for source code, giving an estimation of test coverage. This information is also collected individually for each test framework.

Figure 5 also presents the amount of commits, tags and releases, giving their monthly and daily mean, along with the mean time between occurrences. For instance, for a total of 8497 commits, there are on average 4 per day, but only 2 releases per month. For this particular repository, the GitHub API did not return any release, although, the GitHub releases page shows that there are 154 releases by the time of this analysis, the same number of tags. We understand it as a failure in the API.

For the issues, it was found a total 6673, in which 974 were classified as bug reports. Also, the tool could not classify accurately any issues as features. Figure 4 also indicates there is a greater percentage of open issues classified as bugs when compared to the number of all open issues. Also, the average Lead Time of the issues classified as bugs were significantly bigger than the average of all the issues. This may indicate that some bugs were not treated as high-priority issues and remained for too long in the backlog.

4 COMPARING REPOSITORIES

This sections present how Garimpeiro can be used to compare repositories. For this, we used the tool to extract data from two repositories, Exoplayer and Litho.

Garimpeiro

google/ExoPlayer

Download JSON Report

Contributors

Number of contributors: 144

Number of contributions: 6681

Badges

No badges to Display.

Topics

android mediaplayer java exoplayer

CI	Build	Artifact Integration	Unit testing	Acceptance Testing	UI testing
No tools found.	gradle	No tools found.	junit (22.33%) mockito (2.92%)	No tools found.	No tools found.

Tests

Modules: 891 Test Files: 199 Test Coverage: 22,33%

	Unit Testing	Acceptance Testing	UI Testing
Files	199.0	0.0	0.0%
Coverage	22,33%	0%	0%

Frequency

	Commits: 8497.0			Tags: 154.0			Releases: 0.0		
	Time Between	Per Day	Per Month	Time Between	Per Day	Per Month	Time Between	Per Day	Per Month
Mean	0d 5h 36m	4	130	11d 4h 25m	0	2	0d 0h 0m	0	0
Standard Deviation	0d 0h 11m	106,86	37,93	0d 0h 1m	∞	9,17	0d 0h 0m	0	0

Issues

	General	Open	Closed	Lead Time (Mean)
All	6673.0	358.0 (5,36%)	6315.0 (94,64%)	40d 22h 28m
Bugs	974.0	78.0 (8,01%)	896.0 (91,99%)	77d 11h 34m
Features	0.0	0.0 (0%)	0.0 (0%)	0d 0h 0m

Issues Labels

needs triage question cla: yes bug need more info wont fix: infeasible enhancement wont fix: intended behavior duplicate bug: device specific bad media should merge low priority documentation candidate stale wont fix: not reproducible closed-by-bot wont fix: app issue cla: no content not playing bug: in platform drm bug: in cast receiver bug: in dependency bug: in platform wont merge

Badges Details

No badges to Display.

Figure 5: Garimpeiro output for the ExoPlayer repository.

We understand the communities for both repositories have about the same scale based on the number of contributors and contributions. In Table 1, it is possible to compare their tool chains across the stages of their deployment pipelines. Likely, ExoPlayer uses an external CI server, which is not possible to detect in the repository. Both projects are supporting software to develop mobile applications for Android. It justifies the use of gradle as the build automation tool. JUnit and Mockito appear as the unit testing support. No tool could be detected for acceptance or UI tests.

Table 2 compares the data regarding the frequency of commits, tags and releases from both of the repositories. In this example, Litho seems to be more continuous, i.e., it presents new commits and releases more often.

Table 3 shows the number of Java and test files, coverage percentage, as well as the number of test files. Also, it shows both of the repositories use the same test frameworks with almost the same coverage. For these tests, ExoPlayer uses JUnit in all of them (equals of total coverage) and Mockito is responsible for part of it (3.46%).

Finally, Table 4 shows the issue-related data, evidencing the percentage of open and closed issues, as well as their average lead time. For both repositories, no issue is classified as feature. In addition, it presents the number of pull request (PRs). This information can be

Table 1: Tools detected for each pipeline

Project	CI Server	Build	Artifact Repository	Unit Tests	Acceptance Tests	UI Tests
ExoPlayer	-	Gradle	-	JUnit Mockito	-	-
Litho	Circle CI	Gradle	-	JUnit Mockito	-	-

Table 2: Comparing frequency of commits, releases, and tags

	Interval	Per Day	Per Month	Interval	Per Day	Per Month	Interval	Per Day	Per Month
ExoPlayer									
	Commits (10058)			*Tags (166)*			*Releases (-)*		
Mean	5h 13m	4.72	141.66	13d 5h	0.08	2.34	-	-	-
Std. Dev.	11m	104	43.19	1m	59.4	7.22	-	-	-
Litho									
	Commits (10940)			*Tags (88)*			*Releases (41)*		
Mean	2h 34m	9.6	287	13d 4h	0.08	2.38	28d 12h	0.04	1.11
Std. Dev.	16m	1608	284	1m	10.65	5.73	1m	9.01	2.33

Table 3: Comparing test data

Project	Java	Test	Coverage	JUnit	Mockito
ExoPlayer	982	217	22.1%	22.1%	3.46%
Litho	1502	334	22.24%	21.97%	8.46%

Table 4: Comparing issues, bugs, and lead time

	General	Open	Closed	Lead Time	PRs
ExoPlayer					
All	7469	379 (5%)	7090 (94.9%)	42d 1h	6589
Bugs	1118	75 (6.7%)	1043 (93.3%)	83d 3h	1118
Litho					
All	663	73 (11%)	590 (89.%)	63d 17h	323
Bugs	4	2 (50%)	2 (50%)	152d 13h	4

used to triangulate with commits and releases information in order to understand continuity. For instance, looking solely to commits information (Table 2), we perceive Litho as a project running faster as it shows an average of 2 hours and 34 minutes between commits, against 5 hours and 13 minutes for the ExoPLayer project. However, the information about issues shows ExoPlayer has a shorter lead time (about 21 days). Here, we can suppose that internal deliveries may be more frequent in Litho, but in an end-to-end perspective, ExoPlayer delivers more often.

Although comparing repositories in pairs may not be the most efficient way, the JSON report can be used to analyze several repositories using additional processing mechanisms.

5 FINAL REMARKS

This paper showcases how Garimpeiro mines GitHub repositories to provide useful information regarding the application of CI/CD practices in open source repositories. Also, we describe how the tool extract data from GitHub repositories by showing the execution workflow, high-level architecture, and involved assumptions to define metrics.

A running example using the ExoPlayer repository shows the extracted data, characterizing the level of adoption of such practices in the project under several aspects, such as toolchain, frequency of commits and releases, test coverage, and lead time. Also, we use Garimpeiro to compare repositories.

This is the first version of the tool, and some limitations are still in place to be solved. For instance, the number of supported tools.

As future work, we plan to support to a greater number of tools, frameworks and programming languages. Some improvements can be done in the way the tool process some metrics, like improving the calculation of the mean time of commits to take into account the parallelism caused by two or more contributors working in different branches simultaneously. As well as performance improvements, like parallelism for the data gathering process.

REFERENCES

[1] Vincent Driessen. 2010. *A successful Git branching model.* Retrieved June 10, 2020 from URLhttp://nvie.com/posts/a-successful-git-branching-model
[2] Jez Humble and David Farley. 2010. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.* Pearson Education.
[3] Simo Mäkinen, Marko Leppänen, Terhi Kilamo, Anna-Liisa Mattila, Eero Laukkanen, Max Pagels, and Tomi Männistö. 2016. Improving the delivery cycle: A multiple-case study of the toolchains in Finnish software intensive enterprises. *Information and Software Technology* 80 (2016), 175–194.
[4] Mathias Meyer. 2014. Continuous integration and its tools. *IEEE software* 31, 3 (2014), 14–16.
[5] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating GitHub for engineered software projects. *Empirical Software Engineering* 22, 6 (2017), 3219–3253.